

Creating Embedded Systems with Changing Requirements

Take steps toward a successful embedded design that integrates requirement changes as they occur with the least impact to the overall system and schedule

■ JON D. PEARSON

Ask a new or a seasoned embedded systems developer casually how he intends to execute the next project, and the answer will be something like this: "We'll get the requirements, design the system, write the code and then verify it." And the very same day, this coder can be in a conference room at the formative stages of the said project and as soon as he gets a whiff of what the project is about, the code is being written (at least in his head) and the architecture has been locked-in.

The same coder, a day or a week later, can be in a very similar meeting lamenting that there are not enough requirements to do his design and until the requirements stop changing, he cannot proceed. What accounts for this seeming contradiction?

Every project has risk, and the greatest risk is that the requirements will either not be well-defined at the start or will change many times before the code is completed. At the same time, engineers are tasked with "getting the job done" within the constraints of the situation (and doing so safely, again within the constraints of the situation).

Because most commercial projects will be perishable, time is often the key constraint, whether the push is to meet a holiday sales cycle, a key conference or exhibition, beat a competitor to market or simply meet an expected schedule. Since time is always limited, requirements will suffer and therefore change or develop throughout the project.

As a member of the marketing staff, I admit we start projects before



Fig. 1: Aroma ABT-419DNS convection oven

the requirements are set, compounded by our understanding of what is possible, evolving customer desires (which are imperfectly divined at best) and a constantly changing competitive landscape (especially true in a hot new technology area). But at the same time, engineers really thrive on their work, and so often plunge headfirst into a new project.

So for the engineer, the coder, the real challenge is how to successfully begin and complete a project fast enough so that the product generates maximum revenues, knowing that until it actually gets on the market virtually any requirement is subject to change. And, of course, if it is successful, there will be follow-ons and derivatives coming even faster.

The answer is to follow a strategy built to expect and handle change. It starts by expecting everything to change, and then subdividing and separating aspects of the design behind strong interfaces. Thus maintaining the interfaces and the changes underneath do not impact the rest of the design.

Incidentally, this type of programming is very deliberate and does not happen by chance or in an environment where code is simply cranked out as a

reaction to requirements. Certainly, requirements are important, but you can separate the critical structural ones from the features and, with the following design approach, move forward quickly and adapt quickly. The recipe is "start with the structure, establish communications, build-in the timing and enable swappable features."

Step 1: Start with the structure

When discussion of a new project begins, major structural decisions are generally agreed upon, things like what the product is: a toaster, a blade server, a handheld navigation system or a cellphone. From this 'product' definition, a 'system' definition emerges where the big features are fleshed out and the real engineering begins—taking the product and subdividing into smaller systems. This is the point where the structural design of the sub-systems is determined and nailed down. High-level product features affect the marketability of a product as do the detailed features, but this system definition (the sub-system partitioning) doesn't have a strong impact on what the customer specifically experiences, and is therefore fairly safe from major disruption.

What do you include in the structure definition? This is where the host and peripheral devices are chosen (at least the class of devices). These devices are essentially the high-level hardware requirements and represent the first hardware-software partitioning. The structure of an individual sub-system now needs to be defined and designed. The structure is the frame upon which the rest of the ap-